

# Lecture 3: Intro to proofs for algorithms

Michael Dinitz

September 8, 2026  
601.433/633 Introduction to Algorithms  
(Slides by Jessica Sorrell)

# Today

Discuss common proof techniques for algorithms.

- ▶ Inductive arguments (weak, strong)
- ▶ Proof by contradiction
- ▶ Direct proof
- ▶ Loop invariant
- ▶ Proof by contrapositive

We'll demonstrate proof techniques by proving the correctness and running time of sorting algorithms you've seen before.

## Quicksort review

---

### Algorithm Quicksort

Input: array  $\mathbf{A}$  of length  $n$  (assume all elements distinct)

---

- 1: **if**  $n \leq 1$  **then**
  - 2:     **return**  $\mathbf{A}$
  - 3: **end if**
  - 4: Pick some element  $p \in \mathbf{A}$  as the *pivot*
  - 5: Let  $\mathbf{L}$  be the elements less than  $p$ , let  $\mathbf{G}$  be the elements larger than  $p$
  - 6:  $\mathbf{L}' \leftarrow \text{Quicksort}(\mathbf{L})$
  - 7:  $\mathbf{G}' \leftarrow \text{Quicksort}(\mathbf{G})$
  - 8: **return**  $\mathbf{L}' \| p \| \mathbf{G}'$
-

# Strong Induction vs Weak Induction

Strong induction:

- ▶ Prove property holds for a base case.
- ▶ e.g., Quicksort always returns a sorted array for input arrays of size  $n \leq 1$ .

# Strong Induction vs Weak Induction

Strong induction:

- ▶ Prove property holds for a base case.
- ▶ e.g., Quicksort always returns a sorted array for input arrays of size  $n \leq 1$ .
- ▶ Assume inductive hypothesis, that property holds *for all*  $n \leq k$ . Then show that property holds for  $n = k + 1$ .
- ▶ e.g. Assume Quicksort always returns a sorted array for input arrays of size  $\leq k$ . Show it returns a sorted array for input arrays of size  $k + 1$ .

# Strong Induction vs Weak Induction

Strong induction:

- ▶ Prove property holds for a base case.
- ▶ e.g., Quicksort always returns a sorted array for input arrays of size  $n \leq 1$ .
- ▶ Assume inductive hypothesis, that property holds *for all*  $n \leq k$ . Then show that property holds for  $n = k + 1$ .
- ▶ e.g. Assume Quicksort always returns a sorted array for input arrays of size  $\leq k$ . Show it returns a sorted array for input arrays of size  $k + 1$ .

Weak induction:

- ▶ Prove property holds for a base case.

# Strong Induction vs Weak Induction

Strong induction:

- ▶ Prove property holds for a base case.
- ▶ e.g., Quicksort always returns a sorted array for input arrays of size  $n \leq 1$ .
- ▶ Assume inductive hypothesis, that property holds *for all*  $n \leq k$ . Then show that property holds for  $n = k + 1$ .
- ▶ e.g. Assume Quicksort always returns a sorted array for input arrays of size  $\leq k$ . Show it returns a sorted array for input arrays of size  $k + 1$ .

Weak induction:

- ▶ Prove property holds for a base case.
- ▶ Assume inductive hypothesis, that property holds for  $n = k$ . Then show that property holds for  $n = k + 1$ .
- ▶ e.g. Assume Quicksort always returns a sorted array for input arrays of size *exactly*  $k$ . Show it returns a sorted array for input arrays of size  $k + 1$ .

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ .

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ . Quicksort( $\mathbf{A}$ ) returns  $\mathbf{A}$ .

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ . Quicksort( $\mathbf{A}$ ) returns  $\mathbf{A}$ . ✓

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ . Quicksort( $\mathbf{A}$ ) returns  $\mathbf{A}$ . ✓
- ▶ Inductive step: Assume Quicksort( $\mathbf{A}$ ) returns a sorted array for all  $\mathbf{A}$  of length  $\leq n$ . Show it returns a sorted array for all  $\mathbf{A}$  of length  $n + 1$ .

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ . Quicksort( $\mathbf{A}$ ) returns  $\mathbf{A}$ . ✓
- ▶ Inductive step: Assume Quicksort( $\mathbf{A}$ ) returns a sorted array for all  $\mathbf{A}$  of length  $\leq n$ . Show it returns a sorted array for all  $\mathbf{A}$  of length  $n + 1$ .
  - ▶ Pick pivot  $p \in \mathbf{A}$ . Let  $\mathbf{L}$  be the elements less than  $p$ , let  $\mathbf{G}$  be the elements larger than  $p$ .

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ . Quicksort( $\mathbf{A}$ ) returns  $\mathbf{A}$ . ✓
- ▶ Inductive step: Assume Quicksort( $\mathbf{A}$ ) returns a sorted array for all  $\mathbf{A}$  of length  $\leq n$ . Show it returns a sorted array for all  $\mathbf{A}$  of length  $n + 1$ .
  - ▶ Pick pivot  $p \in \mathbf{A}$ . Let  $\mathbf{L}$  be the elements less than  $p$ , let  $\mathbf{G}$  be the elements larger than  $p$ .
  - ▶  $\mathbf{L}$  and  $\mathbf{G}$  are of length  $\leq n$ , so by the inductive hypothesis, Quicksort( $\mathbf{L}$ ) and Quicksort( $\mathbf{G}$ ) return sorted arrays  $\mathbf{L}'$  and  $\mathbf{G}'$ .

# Correctness of Quicksort - (Strong) Inductive Proof

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Base case:  $n \leq 1$ . Quicksort( $\mathbf{A}$ ) returns  $\mathbf{A}$ . ✓
- ▶ Inductive step: Assume Quicksort( $\mathbf{A}$ ) returns a sorted array for all  $\mathbf{A}$  of length  $\leq n$ . Show it returns a sorted array for all  $\mathbf{A}$  of length  $n + 1$ .
  - ▶ Pick pivot  $p \in \mathbf{A}$ . Let  $\mathbf{L}$  be the elements less than  $p$ , let  $\mathbf{G}$  be the elements larger than  $p$ .
  - ▶  $\mathbf{L}$  and  $\mathbf{G}$  are of length  $\leq n$ , so by the inductive hypothesis, Quicksort( $\mathbf{L}$ ) and Quicksort( $\mathbf{G}$ ) return sorted arrays  $\mathbf{L}'$  and  $\mathbf{G}'$ .
  - ▶ Therefore  $\mathbf{L}' \parallel p \parallel \mathbf{G}'$  is sorted.

# Why Strong Induction?

- ▶ A weak inductive hypothesis assumes the desired property holds for  $n = k$ .

# Why Strong Induction?

- ▶ A weak inductive hypothesis assumes the desired property holds for  $n = k$ .
- ▶ A strong inductive hypothesis assumes the desired property holds for all  $n \leq k$ .

# Why Strong Induction?

- ▶ A weak inductive hypothesis assumes the desired property holds for  $n = k$ .
- ▶ A strong inductive hypothesis assumes the desired property holds for all  $n \leq k$ .
- ▶ Quicksort recursively calls itself on  $L$  and  $G$ , which we don't know the size of a priori.
- ▶ In strong induction, we assume that Quicksort is correct for all arrays of size  $\leq k$ , so it doesn't matter what the exact sizes of  $L$  and  $G$  are, because we know they are both  $\leq k$ .

# Correctness of Quicksort - Proof by Contradiction

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

## Correctness of Quicksort - Proof by Contradiction

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Assume there exists at least one array such that Quicksort does not return a sorted array. Let  $\mathbf{A}$  be the smallest such array and let  $n$  be the size of  $\mathbf{A}$ .

# Correctness of Quicksort - Proof by Contradiction

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Assume there exists at least one array such that Quicksort does not return a sorted array. Let  $\mathbf{A}$  be the smallest such array and let  $n$  be the size of  $\mathbf{A}$ .
- ▶ Note  $n \geq 2$ .

# Correctness of Quicksort - Proof by Contradiction

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Assume there exists at least one array such that Quicksort does not return a sorted array. Let  $\mathbf{A}$  be the smallest such array and let  $n$  be the size of  $\mathbf{A}$ .
- ▶ Note  $n \geq 2$ .
- ▶ So Quicksort( $\mathbf{A}$ ) picks a pivot element  $p \in \mathbf{A}$ , defines  $\mathbf{L}$  and  $\mathbf{G}$  as the elements less than  $p$  and the elements greater than  $p$  respectively, and recursively calls Quicksort on  $\mathbf{L}$  and  $\mathbf{G}$  to get  $\mathbf{L}'$  and  $\mathbf{G}'$ .

# Correctness of Quicksort - Proof by Contradiction

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Assume there exists at least one array such that Quicksort does not return a sorted array. Let  $\mathbf{A}$  be the smallest such array and let  $n$  be the size of  $\mathbf{A}$ .
- ▶ Note  $n \geq 2$ .
- ▶ So Quicksort( $\mathbf{A}$ ) picks a pivot element  $p \in \mathbf{A}$ , defines  $\mathbf{L}$  and  $\mathbf{G}$  as the elements less than  $p$  and the elements greater than  $p$  respectively, and recursively calls Quicksort on  $\mathbf{L}$  and  $\mathbf{G}$  to get  $\mathbf{L}'$  and  $\mathbf{G}'$ .
- ▶ By the assumption that  $\mathbf{A}$  is the smallest such array,  $\mathbf{L}'$  and  $\mathbf{G}'$  are sorted.

# Correctness of Quicksort - Proof by Contradiction

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ , Quicksort( $\mathbf{A}$ ) returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Assume there exists at least one array such that Quicksort does not return a sorted array. Let  $\mathbf{A}$  be the smallest such array and let  $n$  be the size of  $\mathbf{A}$ .
- ▶ Note  $n \geq 2$ .
- ▶ So Quicksort( $\mathbf{A}$ ) picks a pivot element  $p \in \mathbf{A}$ , defines  $\mathbf{L}$  and  $\mathbf{G}$  as the elements less than  $p$  and the elements greater than  $p$  respectively, and recursively calls Quicksort on  $\mathbf{L}$  and  $\mathbf{G}$  to get  $\mathbf{L}'$  and  $\mathbf{G}'$ .
- ▶ By the assumption that  $\mathbf{A}$  is the smallest such array,  $\mathbf{L}'$  and  $\mathbf{G}'$  are sorted.
- ▶ Therefore  $\mathbf{L}' \parallel p \parallel \mathbf{G}'$  is sorted.
- ▶ Contradiction: Quicksort( $\mathbf{A}$ ) *does* return a sorted array.

# Direct Proof

A direct proof argues the conclusion of a claim *directly* from its assumptions.

# Direct Proof

A direct proof argues the conclusion of a claim *directly* from its assumptions.

For a statement of the form  $\mathbf{A} \Rightarrow \mathbf{B}$ , a direct proof shows that  $\mathbf{B}$  follows from the logical implications of  $\mathbf{A}$ .

# Quicksort Runtime - Direct Proof

**Claim:** Quicksort runs in time  $O(n^2)$  in the worst case.

# Quicksort Runtime - Direct Proof

**Claim:** Quicksort runs in time  $O(n^2)$  in the worst case.

- ▶ Before making its two recursive calls, Quicksort compares every element of its input array to the pivot, taking time  $\Theta(n)$ .

# Quicksort Runtime - Direct Proof

**Claim:** Quicksort runs in time  $O(n^2)$  in the worst case.

- ▶ Before making its two recursive calls, Quicksort compares every element of its input array to the pivot, taking time  $\Theta(n)$ .
- ▶ The worst case for runtime occurs when the pivot is the smallest or largest element of the array. (slightly informal)

# Quicksort Runtime - Direct Proof

**Claim:** Quicksort runs in time  $O(n^2)$  in the worst case.

- ▶ Before making its two recursive calls, Quicksort compares every element of its input array to the pivot, taking time  $\Theta(n)$ .
- ▶ The worst case for runtime occurs when the pivot is the smallest or largest element of the array. (slightly informal)

$$\begin{aligned} T(n) &= T(|L|) + T(|G|) + \Theta(n) = T(|L|) + T(n - 1 - |L|) + \Theta(n) \\ &\leq T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n) \end{aligned}$$

# Quicksort Runtime - Direct Proof

**Claim:** Quicksort runs in time  $O(n^2)$  in the worst case.

- ▶ Before making its two recursive calls, Quicksort compares every element of its input array to the pivot, taking time  $\Theta(n)$ .
- ▶ The worst case for runtime occurs when the pivot is the smallest or largest element of the array. (slightly informal)

$$\begin{aligned} T(n) &= T(|L|) + T(|G|) + \Theta(n) = T(|L|) + T(n - 1 - |L|) + \Theta(n) \\ &\leq T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n) \end{aligned}$$

- ▶ Solve:  $T(n) = \Theta(n^2)$ .

# Insertion Sort Review

---

**Algorithm** Insertion Sort

---

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do  
2:    $j \leftarrow i$   
3:   while  $j > 1$  and  $A[j] < A[j - 1]$  do  
4:     Swap  $A[j]$  and  $A[j - 1]$   
5:      $j \leftarrow j - 1$   
6:   end while  
7: end for
```

---

# Proof by Loop Invariant (induction)

Proof by loop invariant is a proof technique that establishes some useful property that is true throughout every loop of an iterative algorithm.

- ▶ Initialization: the property is true at the start of the loop.

# Proof by Loop Invariant (induction)

Proof by loop invariant is a proof technique that establishes some useful property that is true throughout every loop of an iterative algorithm.

- ▶ Initialization: the property is true at the start of the loop.
- ▶ Maintenance: if the property is true at the beginning of an iteration, it is true at the beginning of the next iteration.

# Proof by Loop Invariant (induction)

Proof by loop invariant is a proof technique that establishes some useful property that is true throughout every loop of an iterative algorithm.

- ▶ Initialization: the property is true at the start of the loop.
- ▶ Maintenance: if the property is true at the beginning of an iteration, it is true at the beginning of the next iteration.
- ▶ Termination: when the loop terminates, the invariant holds and shows that the algorithm is correct.

# Proof by Loop Invariant (induction)

Proof by loop invariant is a proof technique that establishes some useful property that is true throughout every loop of an iterative algorithm.

- ▶ Initialization: the property is true at the start of the loop.
- ▶ Maintenance: if the property is true at the beginning of an iteration, it is true at the beginning of the next iteration.
- ▶ Termination: when the loop terminates, the invariant holds and shows that the algorithm is correct.

Just induction on time!

## Correctness of Insertion Sort - Proof by Loop Invariant

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ ,  $\text{InsertionSort}(\mathbf{A})$  returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

## Correctness of Insertion Sort - Proof by Loop Invariant

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ ,  $\text{InsertionSort}(\mathbf{A})$  returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Loop invariant: at the beginning of iteration  $i$  (or end of iteration  $i - 1$ ),  $\mathbf{A}[1, i - 1]$  contains all elements of the original input array  $\mathbf{A}[1, i - 1]$ , and is sorted.

# Correctness of Insertion Sort - Proof by Loop Invariant

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ ,  $\text{InsertionSort}(\mathbf{A})$  returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Loop invariant: at the beginning of iteration  $i$  (or end of iteration  $i - 1$ ),  $\mathbf{A}[1, i - 1]$  contains all elements of the original input array  $\mathbf{A}[1, i - 1]$ , and is sorted.
- ▶ Initialization - At the beginning of the first iteration  $i = 2$ ,  $\mathbf{A}[1]$  is sorted.
- ▶ Maintenance - In a single iteration, element  $\mathbf{A}[i]$  of the input array is moved to the left until it is no longer smaller than the element to its left; therefore at the beginning of the next iteration,  $\mathbf{A}[1, i]$  is sorted and contains exactly the same elements as  $\mathbf{A}[1, i]$  from the original input array.

## Correctness of Insertion Sort - Proof by Loop Invariant

**Claim:** Given an array  $\mathbf{A}$  of length  $n$ ,  $\text{InsertionSort}(\mathbf{A})$  returns an array with all elements of  $\mathbf{A}$  sorted from least to greatest.

**Proof:**

- ▶ Loop invariant: at the beginning of iteration  $i$  (or end of iteration  $i - 1$ ),  $\mathbf{A}[1, i - 1]$  contains all elements of the original input array  $\mathbf{A}[1, i - 1]$ , and is sorted.
- ▶ Initialization - At the beginning of the first iteration  $i = 2$ ,  $\mathbf{A}[1]$  is sorted.
- ▶ Maintenance - In a single iteration, element  $\mathbf{A}[i]$  of the input array is moved to the left until it is no longer smaller than the element to its left; therefore at the beginning of the next iteration,  $\mathbf{A}[1, i]$  is sorted and contains exactly the same elements as  $\mathbf{A}[1, i]$  from the original input array.
- ▶ Termination - When the loop terminates,  $i = n + 1$  and therefore  $\mathbf{A}[1, n]$  is sorted and contains exactly the same elements as  $\mathbf{A}[1, n]$  from the original input array. Therefore the original input array has been sorted.

# Proof by Contrapositive

Proof by contrapositive is a proof technique for conditional statements. That is, statements of the form “If  $P$ , then  $Q$ .”

# Proof by Contrapositive

Proof by contrapositive is a proof technique for conditional statements. That is, statements of the form “If  $P$ , then  $Q$ .”

It relies on the fact that  $P \Rightarrow Q$  is logically equivalent to  $\neg Q \Rightarrow \neg P$ .

# Proof by Contrapositive

Proof by contrapositive is a proof technique for conditional statements. That is, statements of the form “If  $P$ , then  $Q$ .”

It relies on the fact that  $P \Rightarrow Q$  is logically equivalent to  $\neg Q \Rightarrow \neg P$ .

To prove  $P \Rightarrow Q$  by contrapositive, we show that if the negation of the conclusion is true ( $\neg Q$ ), then the negation of the hypothesis is true ( $\neg P$ ).

# Correctness of (one iteration of) Insertion Sort - Setup

Example: one iteration of insertion sort.

- ▶ "In a single iteration, element  $x = A[i]$  of the input array is moved to the left until it is no longer smaller than the element to its left; therefore at the beginning of the next iteration,  $A[1, i]$  is sorted and contains exactly the same elements as  $A[1, i]$  from the original input array."
- ▶ **Claim:** if  $x = A[i]$  is moved left until it is larger than the element to its left, then at the end of the iteration  $A[1, i]$  is sorted.

---

**Algorithm** Insertion Sort

---

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j - 1]$ 
4:     do
5:       Swap  $A[j]$  and  $A[j - 1]$ 
6:        $j \leftarrow j - 1$ 
7:   end while
8: end for
```

---

# Correctness of (one iteration of) Insertion Sort - Setup

Example: one iteration of insertion sort.

- ▶ "In a single iteration, element  $x = A[i]$  of the input array is moved to the left until it is no longer smaller than the element to its left; therefore at the beginning of the next iteration,  $A[1, i]$  is sorted and contains exactly the same elements as  $A[1, i]$  from the original input array."
- ▶ **Claim:** if  $x = A[i]$  is moved left until it is larger than the element to its left, then at the end of the iteration  $A[1, i]$  is sorted.
- ▶  $P = "x = A[i] \text{ is moved left until it is larger than the element to its left}."$
- ▶  $Q = "At the end of the iteration  $A[1, i]$  is sorted."$
- ▶ We want to prove  $P \Rightarrow Q$ .

---

**Algorithm** Insertion Sort

---

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j - 1]$ 
     do
4:     Swap  $A[j]$  and  $A[j - 1]$ 
5:      $j \leftarrow j - 1$ 
6:   end while
7: end for
```

---

## Correctness of (one iteration of) Insertion Sort - Setup

- ▶  $P = "x = A[i] \text{ is moved left until it is larger than the element to its left.}"$
- ▶  $Q = "At \text{ the end of the iteration } A[1, i] \text{ is sorted.}"$
- ▶ We want to prove  $P \Rightarrow Q$ .

---

### Algorithm Insertion Sort

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j - 1]$ 
     do
4:     Swap  $A[j]$  and  $A[j - 1]$ 
5:      $j \leftarrow j - 1$ 
6:   end while
7: end for
```

---

## Correctness of (one iteration of) Insertion Sort - Setup

- ▶  $P = "x = A[i] \text{ is moved left until it is larger than the element to its left.}"$
- ▶  $Q = "At \text{ the end of the iteration } A[1, i] \text{ is sorted.}"$
- ▶ We want to prove  $P \Rightarrow Q$ .
- ▶ **Contrapositive** ( $\neg Q \Rightarrow \neg P$ ): If  $A[1, i]$  is *not* sorted at the end of the iteration, then  $x = A[i]$  is *smaller* than the element to its left.

---

### Algorithm Insertion Sort

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j - 1]$ 
     do
4:     Swap  $A[j]$  and  $A[j - 1]$ 
5:      $j \leftarrow j - 1$ 
6:   end while
7: end for
```

---

# Correctness of (one iteration of) Insertion Sort - Proof

**Contrapositive** ( $\neg Q \Rightarrow \neg P$ ): If  $A[1, i]$  is *not* sorted at the end of the iteration, then  $x = A[i]$  is *smaller* than the element to its left.

- Suppose that at end of iteration  $i$ ,  $x$  was moved to location  $j$ .

---

## Algorithm Insertion Sort

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j - 1]$ 
     do
4:     Swap  $A[j]$  and  $A[j - 1]$ 
5:      $j \leftarrow j - 1$ 
6:   end while
7: end for
```

---

# Correctness of (one iteration of) Insertion Sort - Proof

**Contrapositive** ( $\neg Q \Rightarrow \neg P$ ): If  $A[1, i]$  is *not* sorted at the end of the iteration, then  $x = A[i]$  is *smaller* than the element to its left.

- ▶ Suppose that at end of iteration  $i$ ,  $x$  was moved to location  $j$ .
- ▶ Then since insertion sort swaps  $x$  until larger than element to its left, know  $x < A[k]$  after iteration  $i$  for all  $k \in \{j+1, \dots, i\}$ .

---

**Algorithm** Insertion Sort

---

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j-1]$ 
4:     do
5:       Swap  $A[j]$  and  $A[j-1]$ 
6:        $j \leftarrow j-1$ 
7:   end while
8: end for
```

---

## Correctness of (one iteration of) Insertion Sort - Proof

**Contrapositive** ( $\neg Q \Rightarrow \neg P$ ): If  $A[1, i]$  is *not* sorted at the end of the iteration, then  $x = A[i]$  is *smaller* than the element to its left.

- ▶ Suppose that at end of iteration  $i$ ,  $x$  was moved to location  $j$ .
- ▶ Then since insertion sort swaps  $x$  until larger than element to its left, know  $x < A[k]$  after iteration  $i$  for all  $k \in \{j+1, \dots, i\}$ .
- ▶ So if  $A[1, i]$  not sorted by end of iteration, then must be  $A[k]$  with  $k < j$  and  $A[k] > x$ .

---

### Algorithm Insertion Sort

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j-1]$ 
4:     do
5:       Swap  $A[j]$  and  $A[j-1]$ 
6:        $j \leftarrow j-1$ 
7:   end while
8: end for
```

---

# Correctness of (one iteration of) Insertion Sort - Proof

**Contrapositive** ( $\neg Q \Rightarrow \neg P$ ): If  $A[1, i]$  is *not* sorted at the end of the iteration, then  $x = A[i]$  is *smaller* than the element to its left.

- ▶ Suppose that at end of iteration  $i$ ,  $x$  was moved to location  $j$ .
- ▶ Then since insertion sort swaps  $x$  until larger than element to its left, know  $x < A[k]$  after iteration  $i$  for all  $k \in \{j+1, \dots, i\}$ .
- ▶ So if  $A[1, i]$  not sorted by end of iteration, then must be  $A[k]$  with  $k < j$  and  $A[k] > x$ .
- ▶ Since  $A[1, i-1]$  was sorted before iteration (induction), this means that  $A[j-1] > A[k] > x$ .

---

## Algorithm Insertion Sort

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j-1]$ 
4:     do
5:       Swap  $A[j]$  and  $A[j-1]$ 
6:        $j \leftarrow j-1$ 
7:   end while
8: end for
```

---

## Correctness of (one iteration of) Insertion Sort - Proof

**Contrapositive** ( $\neg Q \Rightarrow \neg P$ ): If  $A[1, i]$  is *not* sorted at the end of the iteration, then  $x = A[i]$  is *smaller* than the element to its left.

- ▶ Suppose that at end of iteration  $i$ ,  $x$  was moved to location  $j$ .
- ▶ Then since insertion sort swaps  $x$  until larger than element to its left, know  $x < A[k]$  after iteration  $i$  for all  $k \in \{j+1, \dots, i\}$ .
- ▶ So if  $A[1, i]$  not sorted by end of iteration, then must be  $A[k]$  with  $k < j$  and  $A[k] > x$ .
- ▶ Since  $A[1, i-1]$  was sorted before iteration (induction), this means that  $A[j-1] > A[k] > x$ .
- ▶ So  $x$  is smaller than the element to its left.

---

### Algorithm Insertion Sort

Input: array  $A$  of length  $n$

---

```
1: for  $i \leftarrow 2$  to  $n$  do
2:    $j \leftarrow i$ 
3:   while  $j > 1$  and  $A[j] < A[j-1]$ 
4:     do
5:       Swap  $A[j]$  and  $A[j-1]$ 
6:        $j \leftarrow j-1$ 
7:   end while
8: end for
```

---

Master Theorem from last lecture (time permitting)

# Master Theorem

$$T(n) = aT(n/b) + cn^k \qquad T(1) = c$$

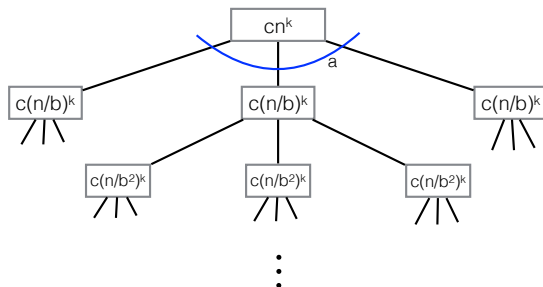
**$a, b, c, k$**  constants with  **$a \geq 1$** ,  **$b > 1$** ,  **$c > 0$** , and  **$k \geq 0$**

# Master Theorem

$$T(n) = aT(n/b) + cn^k$$

$$T(1) = c$$

$a, b, c, k$  constants with  $a \geq 1$ ,  $b > 1$ ,  $c > 0$ , and  $k \geq 0$

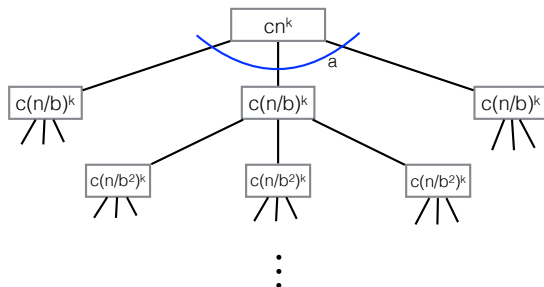


# Master Theorem

$$T(n) = aT(n/b) + cn^k$$

$$T(1) = c$$

$a, b, c, k$  constants with  $a \geq 1$ ,  $b > 1$ ,  $c > 0$ , and  $k \geq 0$



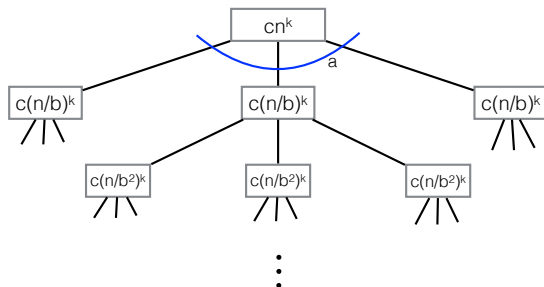
# levels:  $\log_b n + 1$

# Master Theorem

$$T(n) = aT(n/b) + cn^k$$

$$T(1) = c$$

$a, b, c, k$  constants with  $a \geq 1$ ,  $b > 1$ ,  $c > 0$ , and  $k \geq 0$



# levels:  $\log_b n + 1$

Level  $i$ :  $a^{i-1}c(n/b^{i-1})^k = cn^k(a/b^k)^{i-1}$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

- ▶ Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

- ▶ Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$
- ▶ Case 2:  $\alpha < 1$ . Dominated by top level.

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

► Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

► Case 2:  $\alpha < 1$ . Dominated by top level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

$$\implies T(n) = O(n^k)$$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

► Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

► Case 2:  $\alpha < 1$ . Dominated by top level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

$$\implies T(n) = O(n^k)$$

$$T(n) \geq cn^k \implies T(n) = \Omega(n^k)$$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

► Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

► Case 2:  $\alpha < 1$ . Dominated by top level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

$$\implies T(n) = O(n^k)$$

$$T(n) \geq cn^k \implies T(n) = \Omega(n^k) \implies T(n) = \Theta(n^k)$$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

► Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

► Case 2:  $\alpha < 1$ . Dominated by top level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

$$\implies T(n) = O(n^k)$$

$$T(n) \geq cn^k \implies T(n) = \Omega(n^k) \implies T(n) = \Theta(n^k)$$

► Case 3:  $\alpha > 1$ . Dominated by bottom level.

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

► Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

► Case 2:  $\alpha < 1$ . Dominated by top level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

$$\implies T(n) = O(n^k)$$

$$T(n) \geq cn^k \implies T(n) = \Omega(n^k) \implies T(n) = \Theta(n^k)$$

► Case 3:  $\alpha > 1$ . Dominated by bottom level.

$$\begin{aligned} \implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} &= \alpha^{\log_b n} \sum_{i=1}^{\log_b n+1} \left(\frac{1}{\alpha}\right)^{i-1} \leq \alpha^{\log_b n} \frac{1}{1-(1/\alpha)} \\ &= O(\alpha^{\log_b n}) \end{aligned}$$

## Master Theorem II

Let  $\alpha = (a/b^k)$

$$\implies T(n) = cn^k \sum_{i=1}^{\log_b n+1} (a/b^k)^{i-1} = cn^k \sum_{i=1}^{\log_b n+1} \alpha^{i-1}$$

► Case 1:  $\alpha = 1$ . All levels the same.  $T(n) = cn^k \sum_{i=1}^{\log_b n+1} 1 = \Theta(n^k \log n)$

► Case 2:  $\alpha < 1$ . Dominated by top level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

$$\implies T(n) = O(n^k)$$

$$T(n) \geq cn^k \implies T(n) = \Omega(n^k) \implies T(n) = \Theta(n^k)$$

► Case 3:  $\alpha > 1$ . Dominated by bottom level.

$$\implies \sum_{i=1}^{\log_b n+1} \alpha^{i-1} = \alpha^{\log_b n} \sum_{i=1}^{\log_b n+1} \left(\frac{1}{\alpha}\right)^{i-1} \leq \alpha^{\log_b n} \frac{1}{1-(1/\alpha)}$$

$$= O(\alpha^{\log_b n})$$

$$\begin{aligned} \implies T(n) &= \Theta(n^k \alpha^{\log_b n}) = \Theta(n^k (a/b^k)^{\log_b n}) = \Theta(a^{\log_b n}) \\ &= \Theta(n^{\log_b a}) \end{aligned}$$

# Master Theorem III

## Theorem (“Master Theorem”)

*The recurrence*

$$T(n) = aT(n/b) + cn^k$$

$$T(1) = c$$

where  $a, b, c$ , and  $k$  are constants with  $a \geq 1$ ,  $b > 1$ ,  $c > 0$ , and  $k \geq 0$ , satisfies

$$T(n) = \begin{cases} \Theta(n^k) & \text{if } a < b^k, \\ \Theta(n^k \log n) & \text{if } a = b^k, \\ \Theta(n^{\log_b a}) & \text{if } a > b^k. \end{cases}$$